

ui.button 全面详解（基于 NiceGUI 文档）

ui.button 是 NiceGUI 框架中用于创建交互式按钮的核心组件，基于 Quasar 的 QBtn 组件实现，支持丰富的样式定制、事件处理和功能扩展，适用于各类 Web 界面的交互场景。以下从核心特性、使用方法、高级功能等维度进行全面解析。

一、核心基础

1. 组件本质

- 底层依赖：基于 Quasar 的 QBtn 组件，继承了其成熟的样式体系和交互能力。
- 颜色兼容性：支持三种颜色指定方式，优先级为「Quasar 颜色 > CSS 颜色」（若颜色名冲突，如 "red" 同时属于两者，优先使用 Quasar 配色），也可使用 Tailwind 颜色体系。

2. 初始化参数

参数名	类型	说明	默认值
text	str	按钮的文本标签	-
on_click	Callable	按钮点击时触发的回调函数	-
color	str / None	按钮颜色（Quasar/Tailwind/CSS 颜色）	'primary'
icon	str / None	按钮上显示的图标名称（需符合 Quasar 图标规范）	None

二、基础使用示例

1. 最简按钮（带点击反馈）

```
from nicegui import ui

# 点击按钮弹出通知
ui.button('Click me!', on_click=lambda: ui.notify('You clicked me!'))

ui.run()
```

2. 带图标的按钮

支持「仅图标」「文本 + 图标」两种模式，也可嵌套子元素（如图片、标签）：

```
from nicegui import ui

with ui.row():
    # 文本+图标
    ui.button('demo', icon='history')
    # 仅图标（无文本）
    ui.button(icon='thumb_up')
    # 嵌套子元素（文本+图片）
```

```
with ui.button():
    ui.label('sub-elements')
    ui.image('https://picsum.photos/id/377/640/360') \
        .classes('rounded-full w-16 h-16 ml-4')

ui.run()
```

三、关键功能与进阶用法

1. 等待按钮点击（异步流程控制）

通过 `await b.clicked()` 可暂停代码执行，直到按钮被点击后继续，适用于分步操作场景：

```
from nicegui import ui

@ui.page('/')
async def index():
    b = ui.button('Step')
    await b.clicked() # 等待第一次点击
    ui.label('One')

    await b.clicked() # 等待第二次点击
    ui.label('Two')

    await b.clicked() # 等待第三次点击
    ui.label('Three')

ui.run()
```

2. 异步操作时禁用按钮（防止重复点击）

通过上下文管理器 `disable`，在异步任务执行期间自动禁用按钮，任务结束后恢复启用：

```
import httpx
from contextlib import contextmanager
from nicegui import ui

@contextmanager
def disable(button: ui.button):
    button.disable() # 禁用按钮
    try:
        yield
    finally:
        button.enable() # 任务结束后启用

async def get_slow_response(button: ui.button) -> None:
    with disable(button):
        # 模拟耗时请求（延迟 1 秒）
        async with httpx.AsyncClient() as client:
            response = await client.get('https://httpbin.org/delay/1', timeout=5)
            ui.notify(f'Response code: {response.status_code}')
```

```
# 点击按钮触发耗时操作，期间按钮禁用
ui.button('Get slow response', on_click=lambda e: get_slow_response(e.sender))
ui.run()
```

3. 自定义切换按钮（子类扩展）

通过继承 `ui.button` 实现自定义逻辑，例如下方的「红 / 绿状态切换按钮」：

```
from nicegui import ui

class ToggleButton(ui.button):
    def __init__(self, *args, **kwargs) -> None:
        self._state = False # 内部状态（默认关闭）
        super().__init__(*args, **kwargs)
        self.on('click', self.toggle) # 绑定点击切换事件

    def toggle(self) -> None:
        """切换按钮状态"""
        self._state = not self._state
        self.update() # 触发界面更新

    def update(self) -> None:
        # 暂停更新期间修改属性，避免多次渲染
        with self.props.suspend_updates():
            self.props(f'color={"green" if self._state else "red"}')
        super().update()

# 使用自定义切换按钮
ToggleButton('Toggle me')
ui.run()
```

4. 浮动操作按钮（FAB）

通过 `props='fab'` 实现悬浮在页面角落的圆形按钮，常用于核心功能入口：

```
from nicegui import ui

# 设置主题强调色
ui.colors(accent='#6AD4DD')

# 页面粘性定位（右下角）
with ui.page_sticky(x_offset=18, y_offset=18):
    ui.button(icon='home', on_click=lambda: ui.notify('home')) \
        .props('fab color=accent') # fab 属性：圆形+阴影；color 绑定强调色

ui.run()
```

5. 可展开的浮动按钮（多操作入口）

使用 `ui.fab` 和 `ui.fab_action` 组合，实现点击后展开多个子操作的 FAB 组件（基于 Quasar QFab）：

```
from nicegui import ui

with ui.fab():
    ui.fab_action('Add', icon='add', on_click=lambda: ui.notify('Add'))
    ui.fab_action('Edit', icon='edit', on_click=lambda: ui.notify('Edit'))
    ui.fab_action('Delete', icon='delete', on_click=lambda: ui.notify('Delete'))

ui.run()
```

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	元素的 CSS 类（支持 Tailwind/Quasar 类）
enabled	BindableProperty	按钮是否启用（可绑定数据）
html_id	str	HTML 元素的 ID（2.16.0+ 版本支持）
icon	BindableProperty	图标名称（可绑定数据动态修改）
text	BindableProperty	按钮文本（可绑定数据动态修改）
visible	BindableProperty	按钮是否可见（可绑定数据）

2. 关键方法

(1) 状态控制

- `disable()`：禁用按钮
- `enable()`：启用按钮
- `set_enabled(value: bool)`：设置启用状态（True/False）
- `set_visibility(visible: bool)`：设置可见性
- `delete()`：删除按钮及所有子元素

(2) 属性修改

- `set_text(text: str)`：动态修改按钮文本
- `set_icon(icon: str | None)`：动态修改图标
- `update()`：触发客户端界面更新（修改属性后需调用）

(3) 事件与绑定

- `on_click(callback)`: 绑定点击事件 (简化版 `on('click', callback)`)
- `on(type: str, handler)`: 订阅任意事件 (如 'mousedown', 'update:model-value')
- `bind_text(target_object, target_name)`: 双向绑定文本到目标对象的属性
- `bind_icon_from(target_object, target_name)`: 单向绑定图标从目标对象
- `bind_visibility(target_object, target_name)`: 双向绑定可见性

(4) 其他实用方法

- `tooltip(text: str)`: 为按钮添加悬停提示
- `clear()`: 移除所有子元素
- `move(target_container)`: 移动按钮到指定容器
- `mark(*markers)`: 添加标记 (用于测试或元素查询)

五、样式定制

1. 类与样式修改

- 通过 `classes` 叠加样式: `ui.button('Test').classes('p-4 text-lg')` (Tailwind 类: 内边距 4、文本大号)
- 通过 `props` 调整 Quasar 样式: `ui.button('Test').props('outline rounded-lg')` (轮廓样式、大圆角)
- 通过 `style` 直接设置 CSS: `ui.button('Test').style('background: #f00; color: #fff')`

2. 全局默认样式修改

通过 `default_classes`、`default_props`、`default_style` 为所有同类型按钮设置默认样式 (需在实例化前调用):

```
from nicegui import ui

# 设置所有按钮默认使用轮廓样式和大圆角
ui.button.default_props(add='outline rounded-lg')
# 设置默认文本颜色和内边距
ui.button.default_classes(add='text-gray-800 p-3')

ui.button('Button 1')
ui.button('Button 2') # 自动继承默认样式

ui.run()
```

六、使用场景与注意事项

1. 适用场景

- 基础交互：表单提交、页面跳转、操作触发（如删除、保存）
- 分步流程：向导式操作（通过 `await clicked()` 控制步骤）
- 功能入口：浮动按钮（FAB）作为核心功能快捷入口
- 状态切换：自定义子类实现开关、单选等交互逻辑

2. 注意事项

- 颜色优先级：Quasar 颜色与 CSS 颜色同名时，优先使用 Quasar 配色，需自定义颜色时可指定 CSS 十六进制值（如 `color='#ff0000'`）。
- 异步安全：耗时操作必须通过异步函数（`async def`）实现，避免阻塞界面。
- 图标兼容性：图标名称需符合 Quasar 图标库规范（如 'home'、'thumb_up'），不支持自定义图标路径（需通过嵌套 `ui.image` 实现）。
- 绑定机制：双向绑定（如 `bind_text`）会实时同步数据，单向绑定（如 `bind_text_from`）仅从目标对象更新到按钮。